Sistema de Banca por Internet

Contenido

Sistema de Banca por Internet 1

Diagrama de Contexto 3

Diagrama de Contenedores 5

Decisiones Arquitectónicas 7

- Framework Móvil 7
- Framework SPA 7
- Flujo de autenticación OAuth2.0 (Recomendación) 8
- Patron BFF (Backend for Frontend) para la SPA 8
- Onboarding + verificación facial 9
- Método de login post-onboarding + MFA 10
- API Gateway 10
- Notificaciones 11
- BD de auditoría + patrón de caché (Cache-Aside) 12
- Firma de Transacción (Dynamic Linking) 12
- Proveedor de nube (AWS) + mapeo de servicios 13

Diagramas de componentes 14

- Transferencias 14
- Onboarding 14
- Datos Básicos 15
- Movimientos 16
- Notificaciones 16
- Auditoría 17
- Comprobantes 17
- BFF 18

Arquitectura de Acceso a Datos 18

Seguridad 19

Normativa y Cumplimiento 20

Alta Disponibilidad, Tolerancia a Fallos, DR y Monitoreo 21

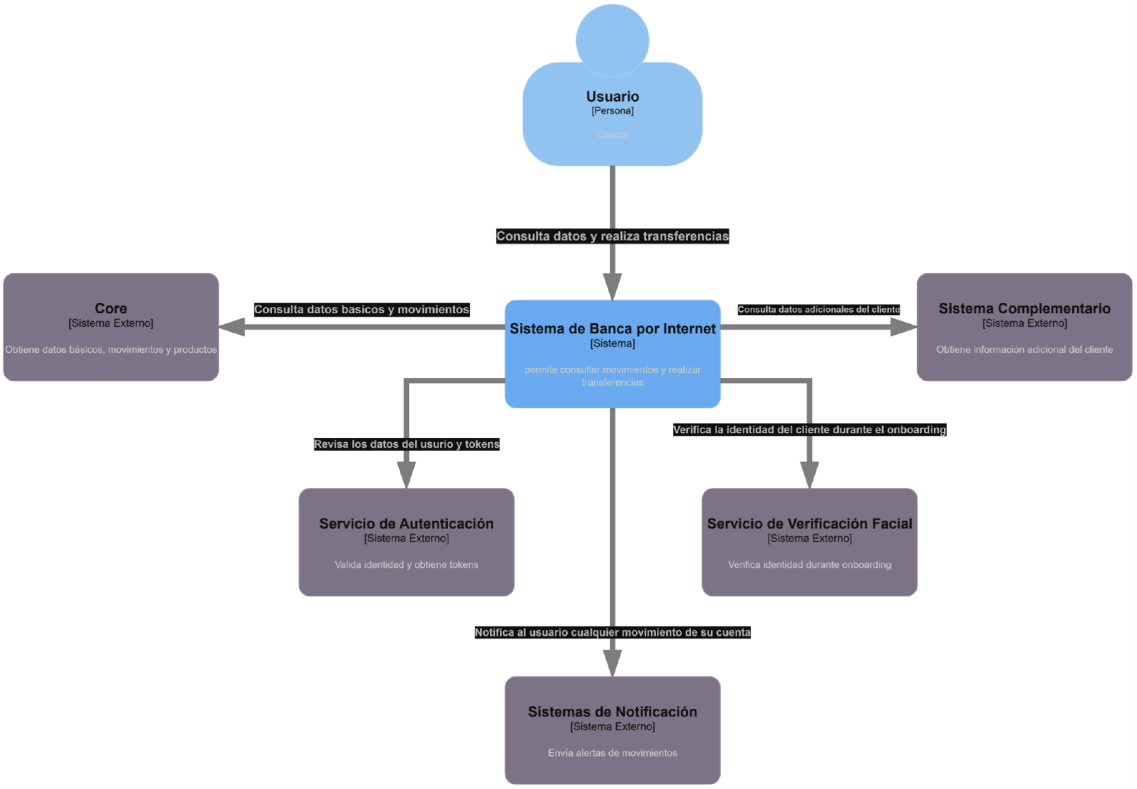
Manejo de Costos 22
Conclusiones 23

Soy Daniel arquitecto de soluciones contratado por una entidad bancaria llamada BP, esta entidad requiere un proyecto para manejar su banca virtual que principalmente sirva para que los clientes revisen su histórico de movimientos, transferencias propias e interbancarias y notificaciones, el cual tiene que ser un sistema con la más alta seguridad y a prueba de fallos.

Para este proyecto se usó un enfoque metodológico C4 para poder definir los componentes tanto en contexto, contenedores y componentes justificando cada decisión arquitectónica tomada.

Este stack está compuesto por 2 aplicaciones en front-end las cuales son un SPA en framework Angular y una app movil realizada en Kotlin Multiplatform, en back-end cuenta con una API Gateway y 7 microservicios internos (Notificaciones, Datos Básicos, Transferencias, Onboarding, Movimientos, Comprobantes y Auditoria) los cuales se conectan a servicios externos como Verificación Facial, Autenticación, Core, Un Sistema Complementario de datos, FCM/APNs, Amazon SES y Proveedor de SMS, con una arquitectura orientada a eventos, desplegada sobre infraestructura de AWS.

Diagrama de Contexto



En el Diagrama de Contexto, el sistema funciona de la siguiente manera: el usuario se conecta al Sistema de Banca por Internet, que procesa todos sus requerimientos.

Para acceder por primera vez, el sistema se conecta con un Servicio de Verificación Facial, que identifica la identidad del usuario y genera su cuenta. A partir de ese momento, el acceso se realiza mediante un Servicio de Autenticación externo, que admite biometría, usuario y contraseña, o PIN.

Cuando el usuario requiere información sobre sus saldos, movimientos, transacciones o datos personales, el sistema se conecta al Core, que cuenta con toda la información necesaria. Si se necesita mayor detalle, el sistema consulta además a un Sistema Complementario.

Por último, cada movimiento se notifica al usuario a través de un Sistema de Notificación.

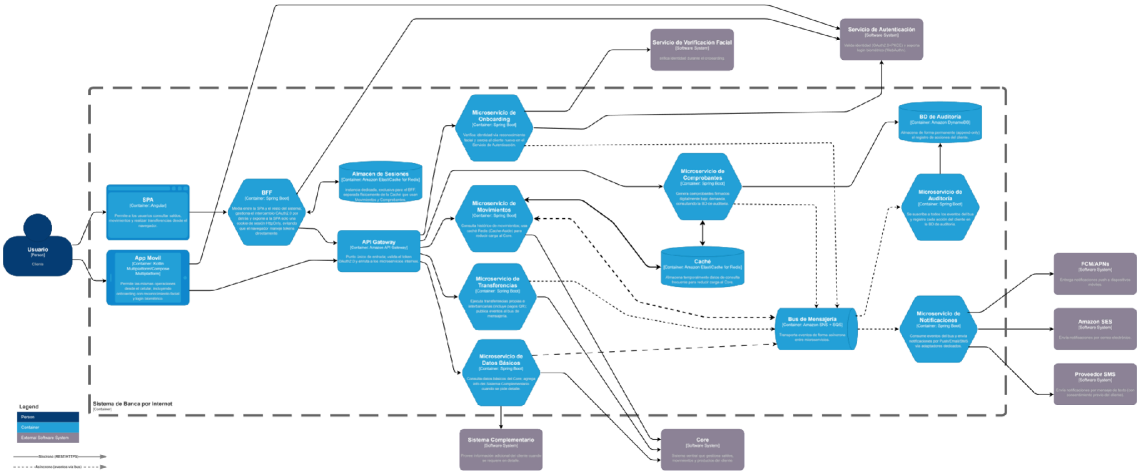
Actor:

Actor	**Descripción**
Usuario/Cliente	Persona que interactúa con el sistema para consultar movimientos, realizar transferencias y pagos

Sistemas externos:

Sistema	**Función**
Core	Obtiene datos básicos, movimientos y productos del cliente
Sistema Complementario	Complementa información adicional del cliente cuando se requiere en detalle
Servicio de Autenticación	Valida identidad y emite tokens de acceso (OAuth2.0 + WebAuthn)
Servicio de Verificación Facial	Verifica identidad del cliente durante el proceso de onboarding
Sistema de Notificación	Notifica al cliente sobre movimientos realizados en su cuenta

Diagrama de Contenedores



Este diagrama de contenedores nos muestra una visión más técnica e interna del sistema de banca por internet donde se puede apreciar todos los contenedores y servicios que lo hacen funcionar, al inicio tenemos 2 apps con las cuales el usuario puede ingresar al sistema, una SPA que por seguridad mediante un BFF (backend for frontend) se comunica con el sistema y una app móvil que, al contar con almacenamiento nativo seguro para los tokens (Keychain/Keystore), puede comunicarse directamente con el sistema sin necesitar esa capa intermedia, tanto el BFF como la app móvil primero tienen que pasar por un sistema externo de autenticación para ingresar.

El sistema usa una API Gateway central para enrutar las peticiones autenticadas hacia los microservicios internos y a su vez estos microservicios se comunican de forma síncrona (REST/HTTPS) con Core/Sistema Complementario, Verificación Facial, Servicio de Autenticación y de forma asíncrona con el Bus de Eventos para los microservicios de Notificaciones, Auditoria y también la Invalidación de cache.

También cuenta con infraestructura de soporte que es un sistema de Cache Redis para consultas, un sistema de Almacén de sesiones que solo usa el BFF para manejar los tokens y secretos del SPA y una Base de Datos de Auditoria donde se manejan todos los eventos del sistema.

Y todo esto esta desplegado sobre AWS que es la que mejor funciona acorde al proyecto.

Contenedor	Tecnología	Descripción
SPA	Angular	Consulta de saldos, movimientos y transferencias desde navegador
App Móvil	Kotlin Multiplatform/Compose Multiplatform	Mismas operaciones desde celular, incluye onboarding facial y login biométrico
BFF	Spring Boot	Media entre SPA y el sistema, gestiona el intercambio OAuth2.0
API Gateway	Amazon API Gateway	Punto único de entrada, valida token, enruta a microservicios
Microservicio de Datos Básicos	Spring Boot	Consulta datos básicos del Core, agrega info del Sistema Complementario
Microservicio de Movimientos	Spring Boot	Consulta histórico de movimientos, usa Cache-Aside
Microservicio de Transferencias	Spring Boot	Transferencias propias e interbancarias, incluye pagos QR
Microservicio de Onboarding	Spring Boot	Verifica identidad facial, enrola cliente nuevo
Microservicio de Notificaciones	Spring Boot	Consume eventos, envía Push/Email/SMS
Microservicio de Auditoría	Spring Boot	Registra todas las acciones del cliente
Microservicio de Comprobantes	Spring Boot	Genera comprobantes firmados bajo demanda
Caché	Amazon ElastiCache for Redis	Datos de consulta frecuente
Almacén de Sesiones	Amazon ElastiCache for Redis (dedicado)	Tokens reales del BFF
Bus de Mensajería	Amazon SNS + SQS	Transporta eventos asíncronos
BD de Auditoría	Amazon DynamoDB	Registro permanente append-only

Decisiones Arquitectónicas

Framework Móvil

Para el framework móvil se consideró 2 opciones React Native y Kotlin Multiplatform (KMP) + Compose Multiplatform entre las cuales se decidió por Kotlin Multiplatform (KMP) + Compose Multiplatform.

Las razones son las siguientes:

1. El rendimiento nativo puro, esto es muy necesario ya que el sistema usa sensores biométricos para el inicio de sesión, y en RN acceder a estos requiere pasar por un puente (bridge) que puede generar cuellos de botella y afectar la experiencia del usuario.
2. Acceso directo a hardware vía expect/actual lo que permite un control completo de hardware.
3. Adopción incremental de módulos, pensando en la arquitectura de implementación de módulos futura esta es una ventaja ya que se puede agregar módulos sin tener que rehacer todo el sistema.
4. Se descarto RN también ya que una ventaja que suele tener este framework es que tiene actualizaciones OTA que son directas sin tener que pasar por revisión lo cual en este caso es una desventaja ya que por seguridad los sistemas de banca deben pasar por revisión antes de ser implementados.
5. Se reconoce que KMP tiene como contrapartida un ecosistema de librerías más joven (especialmente en iOS) comparado con React Native, pero se consideró que las ventajas de rendimiento y seguridad pesan más para este caso de uso bancario.

Framework SPA

Para SPA se evaluaron React, Angular, Vue y Compose for Web y se decidió por usar Angular por las siguientes razones:

1. Es el framework más usado por empresas grandes especialmente de banca.
2. Usa TypeScript por defecto lo que hace que sea más consistente el trabajo de programación especialmente para equipos grandes y diversos.

3. Se descartó Compose for Web a pesar que era un buen candidato por tener el mismo compose del Multiplatform usado en móvil, es un sistema menos maduro en comparación con el resto de frameworks.
4. React también era una buena opción, pero da más libertad en la programación lo cual puede ser un problema en equipos grandes ya que genera inconsistencias.
5. Vue es más liviano y fácil de aprender, pero tiene menos experiencia en contextos empresariales grandes comparado con Angular.

Flujo de autenticación OAuth2.0 (Recomendación)

Al ser públicos tanto el SPA como la app movil no pueden guardar un client secret de forma segura por lo cual se evaluó cual flujo de autenticación OAuth2.0 recomendar para su mejor funcionamiento, esta es una recomendación ya que la compañía cuenta con su propio producto que puede ser configurable.

Se evaluó entre Implicit Flow, Resource Owner Password Credentials, Client Credentials y Authorization Code + PKCE, entre las cuales se decidió por esta última (Authorization Code + PKCE).

Las Razones son:

1. Envía un secreto temporal aleatorio por cada intento de login lo cual hace casi imposible interceptar
2. Funciona tanto para SPA como para la app móvil
3. Esta recomendado por el propio estándar OAuth 2.0 Security BCP y funcionara en el futuro en OAuth2.1
4. Implicit Flow se descartó por que el token queda expuesto en la url y es vulnerable.
5. También se descartó Resource Owner Password Credentials ya que maneja las contraseñas crudas y es muy fácil de interceptar.
6. Client Credentials este solo aplica para conexiones de servidor-a-servidor así que no aplica.

Patron BFF (Backend for Frontend) para la SPA

Se decidió usar BFF (Backend for Frontend) ya que la SPA nunca debe manejar tokens OAuth2.0 directamente en el navegador (riesgo de robo vía XSS); el BFF media entre la

SPA y el resto del sistema para manejar los tokens, y entrega a la SPA solo una cookie de sesión HttpOnly+Secure. Esta decisión se respalda en las recomendaciones del borrador IETF “OAuth 2.0 for Browser-Based Apps”. Aplica únicamente a la SPA — la App Móvil no usa BFF, ya que puede guardar tokens de forma segura en almacenamiento nativo (Keychain/Keystore).

Para almacenar los tokens reales del lado del servidor, se consideró reutilizar la Caché Redis ya existente, pero se descartó por riesgo de seguridad: mezclar tokens de sesión con datos de consulta cacheados en el mismo almacén aumentaría el daño ante una brecha, dificultaría el control de acceso granular, y expondría los tokens a ser expulsados por presión de memoria en vez de expirar de forma controlada. Por eso se optó por un Almacén de Sesiones dedicado (instancia separada de Amazon ElastiCache for Redis), exclusivo para el BFF.

Onboarding + verificación facial

Este microservicio se usa solamente cuando el usuario es nuevo y está creando su cuenta por primera vez primero la app móvil captura rostro (+liveness) y envía al Servicio de Verificación Facial para que lo analice si es exitoso el Onboarding crea la cuenta y la enrola en el Servicio de Autenticación.

Aquí se ha tomado la decisión de enrutamiento de que la petición entre primero por el API Gateway. Aunque es un usuario nuevo sin token aún, el Gateway, para esta ruta específica, aplica rate limiting y logging sin exigir validación de token OAuth2.0 (ya que el usuario todavía no tiene uno) — manteniendo un único punto de entrada para todo el tráfico de cliente, sin excepciones que abran una puerta trasera de seguridad. Luego, el Onboarding llama directo al Servicio de Verificación Facial sin volver por el API Gateway, para evitar saltos innecesarios. En este microservicio, como en los demás, se usa un patrón Adapter: si es necesario cambiar de proveedor, solo se hacen cambios en el adaptador, no en el microservicio.

Un detalle importante es que al momento de subir el video para la verificación facial no se sube directamente el binario pesado sino se sube directo a S3 vía URL pre-firmada y se envía esa URL al Servicio de Verificación Facial (práctica estándar de proveedores eKYC reales).

Otro detalle es que este micro servicio debería distinguir entre fallo de comunicación para reintentar y fallo de verificación o rechazo en este último caso no reintenta por posible fraude.

Método de login post-onboarding + MFA

En este método hay una diferencia dependiendo de si el usuario está intentando ingresar por la app móvil o por el SPA.

En el caso de la app móvil el sistema usa FIDO2/WebAuthn con biometría del dispositivo (huella/Face ID), el chip seguro guarda llave privada, servidor solo verifica firma, nunca recibe la huella.

Como fallback el usuario puede usar pin o usuario y contraseña, los cuales no viajan en texto plano por seguridad solo desbloquean una llave localmente guardada.

Se usa FIDO2/WebAuthn porque los servicios de autenticación Keycloak/Auth0/Okta-tipo ya traen soporte integrado además WebAuthn es un estándar de industria en banca (W3C + FIDO Alliance), adoptado por bancos/Google/Apple/Microsoft.

MFA es adicional para la SPA ya que no cuenta con sistemas biométricos garantizados en todos los navegadores, y el único método disponible es usuario y contraseña se recomienda usar TOTP o push de aprobación vía app móvil, reutilizando el sistema de notificaciones y su Adaptador Push ya existentes, sin necesidad de construir infraestructura nueva.

API Gateway

Puerta única de entrada para el sistema desde la SPA (vía BFF) y la App Móvil. Esta valida el token OAuth2.0 centralizadamente y enruta las peticiones hacia los microservicios internos.

Se conecta a los microservicios mediante el protocolo REST/HTTPS síncrono diferente al flujo asíncrono vía bus.

Se conecta a los siguientes microservicios:

- Datos Básicos: Este microservicio consulta al Core por información básica del cliente y si se necesita más información usa el Sistema complementario para agregar información mas detallada y lo junta en una sola respuesta mediante patrón Aggregator
- Movimientos: Este microservicio también consulta al core por información, pero para datos mas recurrentes usa Cache-Aside con redis.
- Transferencias: valida reglas de negocio (saldo suficiente, límites) y ejecuta transferencias propias e interbancarias contra el Core; al completarse, publica

un evento al Bus de Mensajería en vez de notificar o auditar directamente, manteniendo el desacoplamiento ya establecido en el resto de la arquitectura.

- Pagos QR: esta funcionalidad se consolidó dentro de Transferencias, ya que realiza la misma acción de fondo y evitamos fragmentar innecesariamente; sigue el estándar internacional EMV QR Code Specification.
- Comprobantes: este microservicio genera los comprobantes seguros y verificables para el uso del cliente, solamente genera bajo demanda (cuando el cliente lo pide) para no desperdiciar almacenamiento a escala bancaria; lee directamente de la DB de Auditoria para no duplicar datos, usa Cache para guardar el comprobante por un tiempo limitado para no generarlo de nuevo en poco tiempo, tiene 3 capas de seguridad complementarias (Defense in Depth): firma digital, QR de verificación, marca de agua.

Notificaciones

Este microservicio consume los eventos del Bus de Mensajería usando un patrón Pub/Sub desacoplado. Varios microservicios (Transferencias, Datos Básicos, Movimientos, Onboarding, Comprobantes) publican eventos y continúan su camino sin esperar respuesta, lo cual evita bloqueos si algo falla en la notificación.

El microservicio de Notificaciones, por su parte, solo se suscribe a los eventos marcados como notificables, para no procesar todos los eventos del bus de mensajería.

También usa un patrón Adapter con un adaptador por canal elegido para notificar: PushAdapter (FCM/APNs), EmailAdapter (Amazon SES), SMSAdapter (SMS solo con consentimiento previo del cliente, por normativa de protección de datos).

Las Razones de esta decisión son:

1. Al estar desacoplado, no se bloquea, si un proveedor tarda o falla no afecta a la transacción principal del usuario.
2. Al ser 3 canales separados por su adaptador si un adaptador falla no tumba el resto de canales así no se necesita 3 microservicios separados.
3. Escalabilidad futura: si un canal se vuelve cuello de botella, se puede separar a su propio microservicio sin rediseñar todo
4. Consistencia de patrones: reutiliza el mismo patrón Adapter ya aplicado en la integración con el Servicio de Verificación Facial y en el push de aprobación de MFA, manteniendo coherencia arquitectónica en todo el sistema.

BD de auditoría + patrón de caché (Cache-Aside)

La base de datos de auditoría reutiliza el mismo Bus de Mensajería que se usa para notificaciones, pero con una suscripción distinta, mientras Notificaciones solo se suscribe a eventos notificables, Auditoría tiene una suscripción amplia (wildcard) a todos los eventos del sistema, esto incluye consultas, logins, fallos y transacciones exitosas o fallidas, para tener trazabilidad completa de cada acción del cliente y no solo de lo que se notifica. Los eventos los genera el propio backend sin depender de que el cliente tenga conexión en ese momento y la base de datos es append-only, solo se insertan registros, nunca se modifican ni se borran, esto garantiza que el historial de auditoría sea inmutable.

Un detalle importante es que los eventos de transacciones deben llevar los datos completos de la operación y no solo un identificador, esto se conoce como Event-Carried State Transfer, gracias a esto el microservicio de Comprobantes puede generar un comprobante leyendo directo de la base de auditoría sin volver a consultar al Core que ya es la dependencia más crítica del sistema, además al ser permanente se pueden generar comprobantes de transacciones antiguas que el Core ya no conserve.

Para clientes frecuentes se implementó Cache-Aside con Redis, el microservicio de Movimientos primero consulta la caché y si no está (miss) consulta al Core y guarda el resultado con un TTL corto, se usa política LRU lo que hace que los datos de clientes que consultan seguido se queden solos en memoria sin necesitar lógica extra para detectar quién es frecuente. La invalidación está conectada al mismo bus de eventos, cuando ocurre una transacción se invalida la entrada de caché correspondiente automáticamente.

Las razones de usar Cache-Aside son: evita sobrecargar al Core con consultas repetidas, mejora el tiempo de respuesta y la experiencia del usuario, y la invalidación basada en eventos asegura datos frescos sin sacrificar la velocidad.

Firma de Transacción (Dynamic Linking)

Antes de confirmar una transferencia se genera un código único atado criptográficamente a los datos exactos de la operación (cuenta destino y monto), este código lo genera el Generador de Firma de Transacción dentro de Transferencias usando Java Cryptography Architecture (JCA).

La idea es que, si alguien alterara los datos en el camino, por ejemplo, un ataque man-in-the-browser que cambia la cuenta destino, el código ya no coincidiría con los datos reales y el sistema rechaza la operación.

Esto se basa en el concepto de Strong Customer Authentication con Dynamic Linking de la regulación europea PSD2, aunque el sistema no opera bajo esa jurisdicción es una práctica de seguridad reconocida mundialmente en banca digital y se decidió aplicarla igual ya que refuerza la prioridad de seguridad del proyecto.

Proveedor de nube (AWS) + mapeo de servicios

Se evaluó entre AWS y Azure para desplegar la infraestructura y se decidió por AWS por dos razones, tiene mayor adopción y reputación en el mercado sobre todo en banca digital, y cuenta con un catálogo más amplio de servicios gestionados.

Cada pieza de la arquitectura tiene su equivalente en AWS: EKS para el orquestador de microservicios, Amazon API Gateway para el Gateway, Amazon SNS + SQS para el bus de mensajería, Amazon ElastiCache for Redis para la caché y el almacén de sesiones (en instancias separadas por seguridad como ya se justificó), Amazon DynamoDB para la BD de auditoría, AWS Secrets Manager para gestión de secretos, CloudWatch con X-Ray para monitoreo y tracing, y AWS WAF para seguridad perimetral.

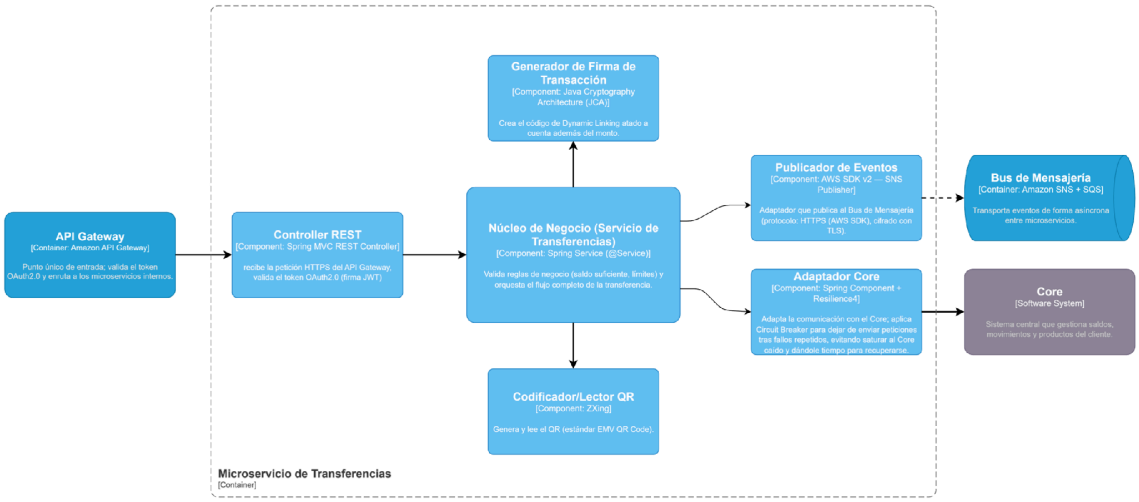
Para la baja latencia se eligió la región de São Paulo, la más cercana a Ecuador y Sudamérica disponible en AWS, ya que la latencia depende de la distancia real entre servidores y usuarios finales y no de dónde esté la sede corporativa. Como optimización extra se puede usar AWS Global Accelerator que enruta el tráfico por la red privada de alta velocidad de AWS en vez de la internet pública.

El Servicio de Autenticación se documenta como un producto empresarial de IdP ya existente en la compañía (tipo Keycloak, Auth0 u Okta) configurado para OAuth2.0 y WebAuthn, auto-hospedado dentro del clúster de EKS, esto es consistente con lo que dice el enunciado sobre que la compañía ya cuenta con un producto configurable para este fin.

Para el manejo de costos la estrategia combina instancias reservadas para la carga base constante junto con auto-scaling para los picos de demanda, así se evita pagar de más por tener todo sobreaprovisionado todo el día al nivel de la hora pico.

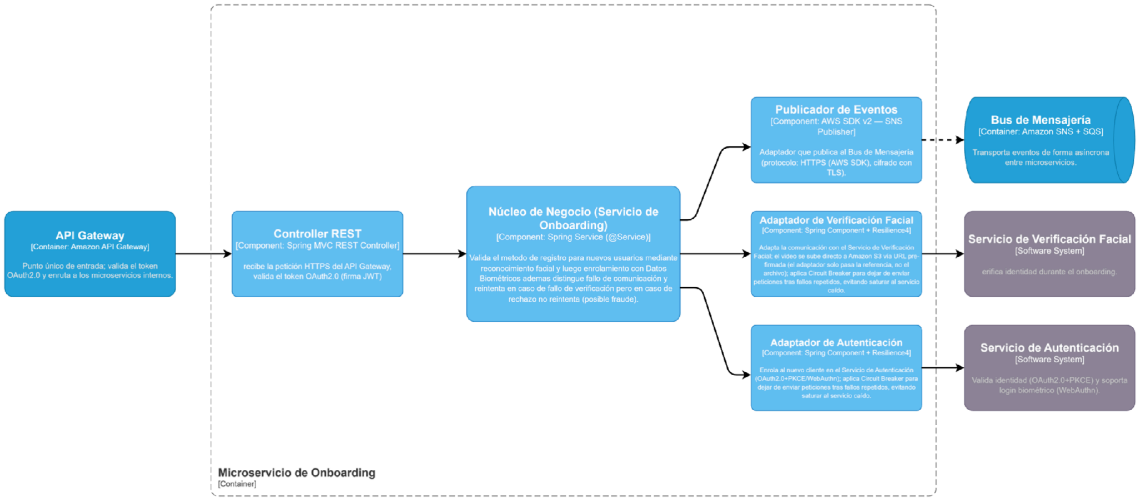
Diagramas de componentes

Transferencias



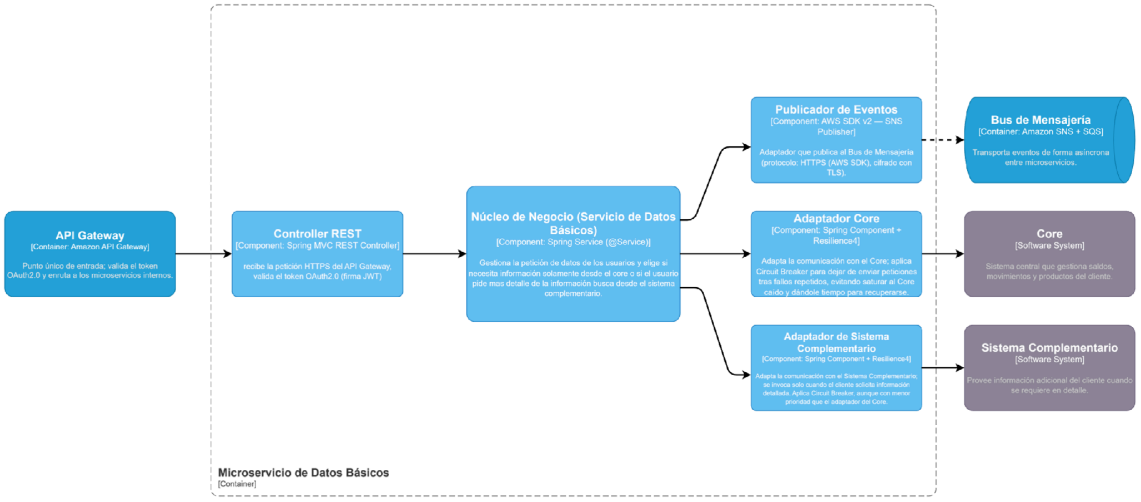
Sigue el patrón de Arquitectura Hexagonal (Ports & Adapters), con el núcleo de negocio en el centro validando reglas (saldo suficiente, límites) y orquestando el flujo. Se apoya en dos componentes auxiliares, el Generador de Firma de Transacción (usando Java Cryptography Architecture) que crea el código de Dynamic Linking, y el Codificador/Lector QR (usando ZXing) para los pagos consolidados. La comunicación hacia el Core se protege con un Adaptador que implementa Circuit Breaker mediante Resilience4j, vía HTTPS/mTLS por tratarse de datos financieros sensibles, mientras que la publicación de eventos hacia el Bus de Mensajería se hace de forma asíncrona vía TLS.

Onboarding



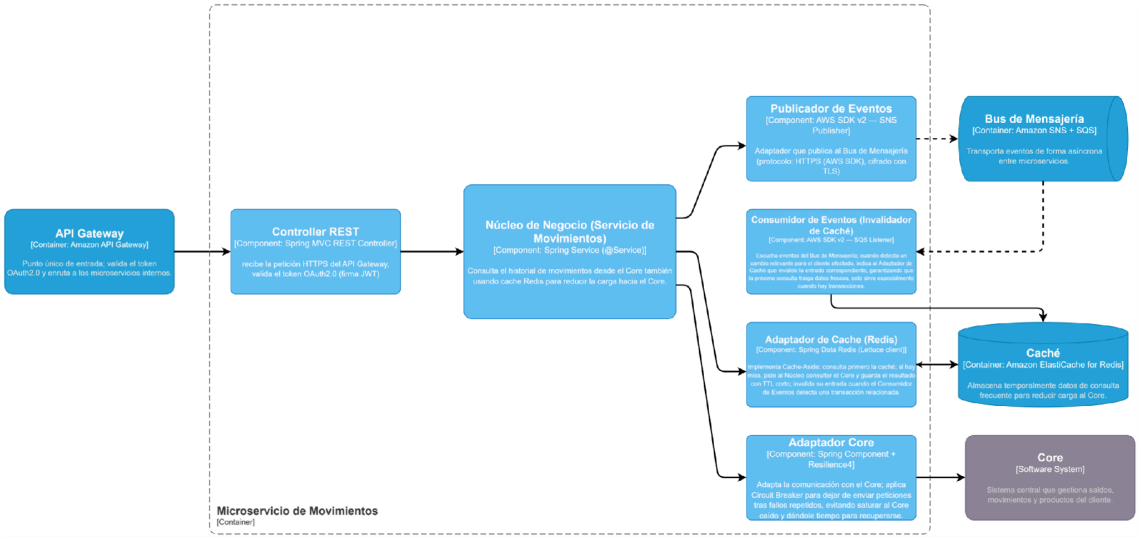
El núcleo de negocio distingue entre fallo de comunicación (reintenta) y fallo de verificación o rechazo (no reintenta, por posible fraude). Se conecta a dos adaptadores con Circuit Breaker, uno hacia el Servicio de Verificación Facial (donde el video se sube directo a Amazon S3 vía URL pre-firmada, evitando sobrecargar el microservicio con tráfico binario pesado) y otro hacia el Servicio de Autenticación para enrolar al cliente nuevo usando OAuth2.0+PKCE/WebAuthn. También publica eventos de auditoría al Bus de Mensajería.

Datos Básicos



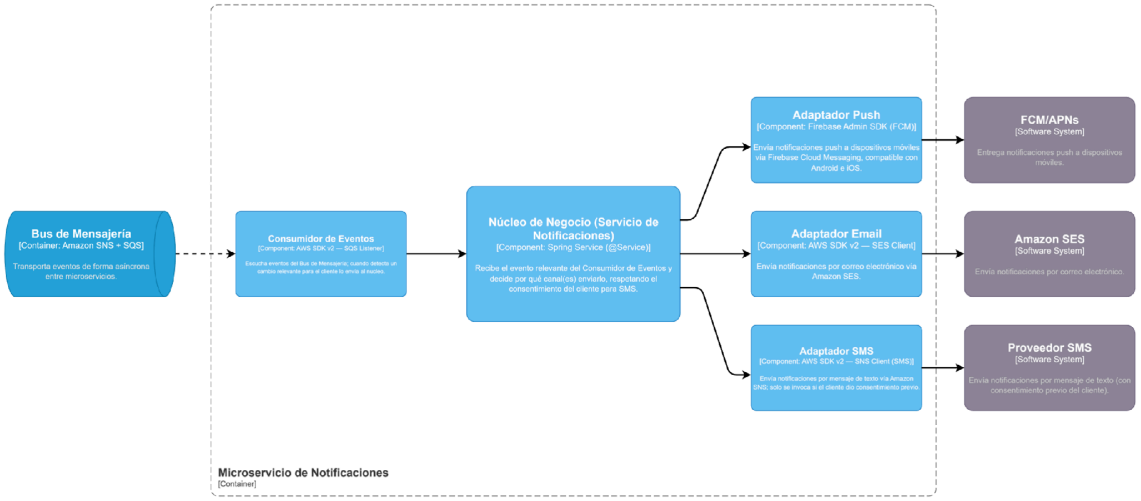
Aquí se implementa el patrón Aggregator, el núcleo de negocio decide si consulta solo al Core o si también necesita consultar al Sistema Complementario cuando se pide información detallada, combinando ambas respuestas en una sola antes de devolverla. Ambos adaptadores usan Circuit Breaker vía Resilience4j, aunque el del Sistema Complementario tiene menor prioridad que el del Core por su menor radio de impacto si falla.

Movimientos



El componente más complejo de los 8 porque pública y consume eventos del bus al mismo tiempo, por eso se separó en dos componentes distintos, el Publicador de Eventos (SNS Publisher) y el Consumidor de Eventos (SQS Listener), este último es un punto de entrada independiente que invalida la caché cuando detecta una transacción relacionada. El Adaptador de Caché usa Spring Data Redis (cliente Lettuce) e implementa el patrón Cache-Aside completo, y el Adaptador Core usa Circuit Breaker igual que en los demás microservicios.

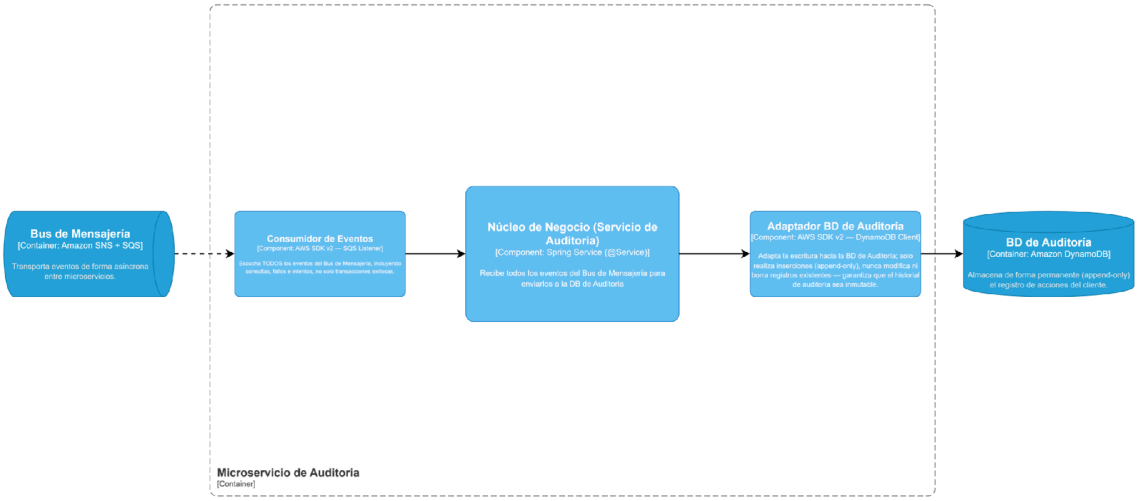
Notificaciones



Un Consumidor de Eventos (SQS Listener) con suscripción selectiva solo a eventos notificables alimenta al núcleo de negocio, que decide por cuál canal enviar respetando el consentimiento del cliente para SMS. Usa el patrón Adapter con tres adaptadores

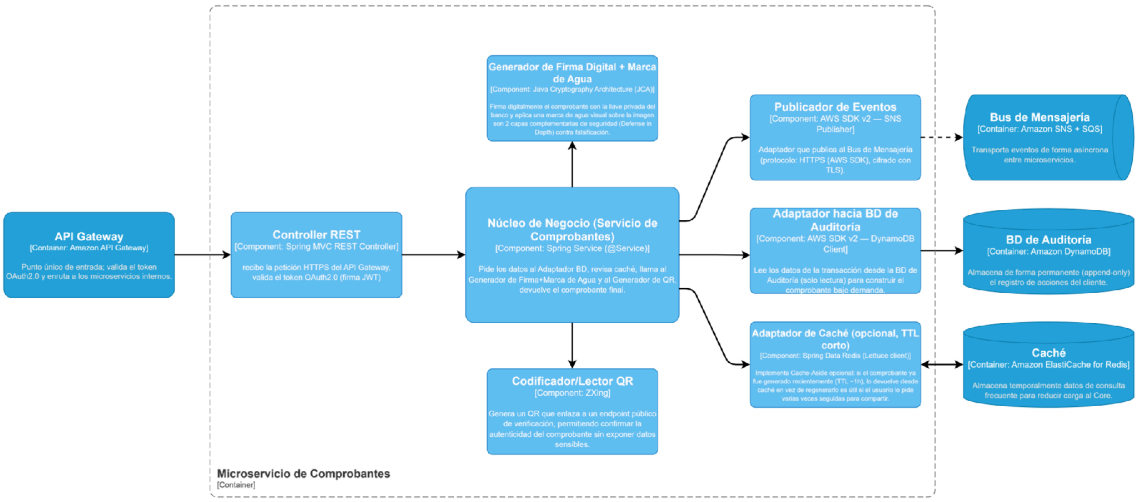
independientes, Push (Firebase Admin SDK/FCM), Email (Amazon SES) y SMS (Amazon SNS), así si un canal falla no afecta a los otros.

Auditoría



Un Consumidor de Eventos con suscripción amplia (wildcard) a todos los temas del bus, sin filtrar nada, alimenta al núcleo que normaliza cada evento antes de insertarlo en la BD de Auditoría mediante el cliente de AWS SDK para DynamoDB, que solo realiza inserciones, nunca modifica ni borra registros existentes.

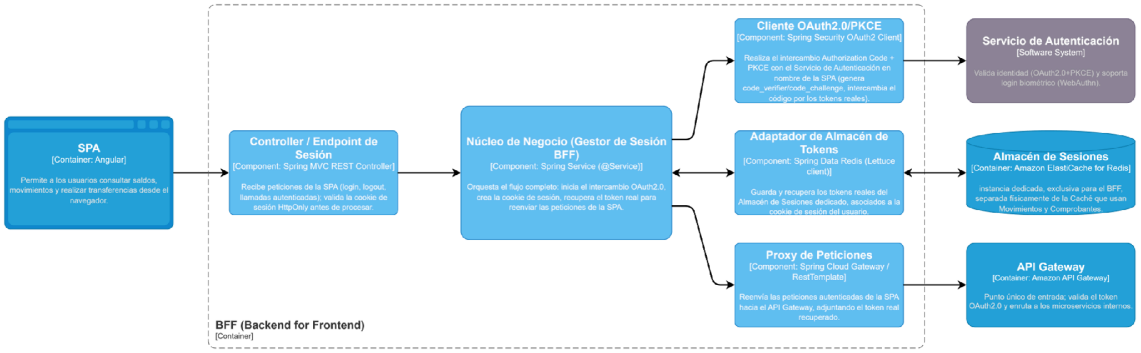
Comprobantes



El Controller recibe el ID de una transacción bajo demanda, el núcleo pide los datos a un Adaptador de solo lectura hacia la BD de Auditoría, y genera el comprobante usando el Generador de Firma Digital + Marca de Agua (JCA) y el Generador/Lector de

QR (ZXing), aplicando las tres capas de Defense in Depth ya justificadas. Opcionalmente usa un Adaptador de Caché con TTL corto por si el comprobante se pide varias veces seguidas para compartir.

BFF



El Gestor de Sesión en el núcleo orquesta todo el flujo, el Cliente OAuth2.0/PKCE realiza el intercambio Authorization Code + PKCE con el Servicio de Autenticación en nombre de la SPA, el Adaptador de Almacén de Tokens guarda y recupera los tokens reales del Almacén de Sesiones dedicado (separado de la caché general por seguridad), y el Proxy de Peticiones reenvía las llamadas autenticadas de la SPA hacia el API Gateway adjuntando el token real.

Arquitectura de Acceso a Datos

El acceso a datos del cliente se centraliza en 2 fuentes externas, el Core que tiene la información básica, movimientos y productos, y el Sistema Complementario que agrega información adicional solo cuando se requiere en detalle. Para combinar ambas fuentes en una sola respuesta se implementó el patrón Aggregator en el microservicio de Datos Básicos, así las aplicaciones cliente (SPA y App Móvil) nunca saben que por detrás hubo 2 llamadas distintas a 2 sistemas distintos.

Todas las llamadas hacia el Core y el Sistema Complementario son síncronas vía REST/HTTPS y están protegidas con el patrón Circuit Breaker (usando Resilience4j), priorizado según el radio de impacto de cada sistema, el Core tiene la máxima prioridad porque lo usan 3 microservicios (Datos Básicos, Movimientos, Transferencias) y afecta a todos los clientes existentes si falla, mientras que el Sistema Complementario tiene menor prioridad porque su uso es ocasional y su impacto más acotado.

Para reducir la carga al Core en consultas repetidas se implementó el patrón Cache-Aside con Redis en el microservicio de Movimientos, con política de expulsión LRU que mantiene en memoria los datos de los clientes frecuentes sin necesitar lógica extra para detectarlos, y con invalidación automática conectada al Bus de Mensajería cada vez que ocurre una transacción relacionada.

Como fuente adicional de datos históricos está la BD de Auditoría, que al guardar los eventos completos de cada transacción (Event-Carried State Transfer) permite que el microservicio de Comprobantes genere comprobantes de transacciones antiguas sin necesidad de volver a consultar al Core.

Seguridad

La seguridad del sistema se construyó en varias capas complementarias, siguiendo el principio de Defense in Depth, no se depende de un solo mecanismo sino de varios trabajando juntos.

Capa de autenticación: tanto la SPA como la App Móvil usan el flujo OAuth2.0 Authorization Code + PKCE, el más recomendado por el estándar para clientes públicos. La SPA además protege sus tokens con un patrón BFF, el navegador nunca maneja tokens OAuth2.0 directamente, solo recibe una cookie de sesión HttpOnly+Secure, y los tokens reales se guardan en un Almacén de Sesiones dedicado, separado de la caché general por motivos de seguridad. Para el método de login, la App Móvil usa FIDO2/WebAuthn con la biometría del dispositivo (la llave privada nunca sale del Secure Enclave/TEE del celular) con PIN como respaldo, mientras que la SPA usa usuario y contraseña ya que no se garantiza hardware biométrico en todos los navegadores, reforzado con un segundo factor (MFA) vía TOTP o push de aprobación usando la App Móvil como dispositivo de confianza.

Capa de protección de transacciones: cada transferencia se protege con una Firma de Transacción (Dynamic Linking) que ata criptográficamente el código generado a los datos exactos de la operación, cuenta destino y monto, evitando que un ataque intercepte y modifique la transacción sin que el sistema lo detecte. Los comprobantes generados también se protegen con 3 capas complementarias, firma digital, QR de verificación y marca de agua, dificultando su falsificación.

Capa de protección de datos e identidad: el onboarding de clientes nuevos usa reconocimiento facial, con el video subido directo a Amazon S3 vía URL pre-firmada en vez de pasar por el microservicio, y el núcleo de negocio distingue entre fallos de comunicación transitorios (reintenta) y fallos de verificación o rechazo (no reintenta, por posible fraude).

Capa de red e infraestructura: solo el API Gateway está expuesto públicamente, todos los microservicios y bases de datos viven en una red privada (VPC) sin acceso directo desde internet. Un WAF (AWS WAF) filtra ataques comunes como SQL injection o XSS antes de que lleguen al Gateway. Toda la comunicación entre servicios y hacia sistemas externos sensibles usa HTTPS, con mTLS para el tráfico hacia el Core, y la gestión de secretos y credenciales se hace mediante AWS Secrets Manager en vez de guardarlos directo en el código.

Toda esta arquitectura de seguridad se respalda en normativa y estándares reconocidos de la industria, detallados en la siguiente sección.

Normativa y Cumplimiento

Para este proyecto se decidió combinar normativa local (asumiendo Ecuador como país de operación, ya que el enunciado no lo especifica) con estándares internacionales de la industria bancaria, bajo la postura de que una entidad financiera debe cumplir el estándar más alto posible, no solo el mínimo legal exigido.

Normativa/Estándar	**Conecta con**	**Por qué**
LOPDP (Ley Orgánica de Protección de Datos Personales, Ecuador 2021)	Consentimiento previo para notificaciones SMS	Exige consentimiento explícito para el tratamiento de datos personales
LOPDP	Acceso a Core/Sistema Complementario y BD de Auditoría	Principio de minimización y confidencialidad de datos personales
Normativa de la Superintendencia de Bancos del Ecuador	Alta Disponibilidad, DR y Monitoreo	Exige gestión de riesgo tecnológico, continuidad del negocio y gestión de incidentes
Normativa de la Superintendencia de Bancos	BD de Auditoría	Sirve como evidencia y trazabilidad ante incidentes o auditorías regulatorias
PCI-DSS	Comunicación cifrada HTTPS/mTLS entre	Protección de datos financieros en tránsito y en

Normativa/Estándar	Conecta con	Por qué
	servicios, Core y Sistema Complementario	reposo, estándar de facto en la industria
ISO/IEC 27001	Toda la arquitectura de seguridad en capas (Defense in Depth)	Marco de gestión de seguridad de la información reconocido internacionalmente
OWASP ASVS	Descarte de Implicit Flow y Resource Owner Password Credentials, biometría en Secure Enclave/TEE	Estándar específico de seguridad para desarrollo de aplicaciones SPA y móviles
SWIFT CSP (Customer Security Programme)	Microservicio de Transferencias, operaciones interbancarias	Estándar de seguridad específico para integridad de mensajería en operaciones interbancarias

Alta Disponibilidad, Tolerancia a Fallos, DR y Monitoreo

Para garantizar alta disponibilidad, cada microservicio corre en múltiples instancias repartidas en al menos 2 o 3 zonas de disponibilidad (AZ) dentro de la región de AWS, detrás de un balanceador de carga, así si una instancia o una zona completa falla, las demás siguen respondiendo sin interrupción para el usuario.

Para la tolerancia a fallos se implementó el patrón Circuit Breaker (junto con Retry y Timeout) en todas las llamadas hacia sistemas externos, priorizado según el radio de impacto de cada uno y no solo por qué tan confiable es. El Core tiene la máxima prioridad porque lo usan 3 microservicios (Datos Básicos, Movimientos, Transferencias) y afecta a todos los clientes existentes si falla, el Servicio de Verificación Facial tiene prioridad media porque su impacto se limita al onboarding de clientes nuevos, y el Sistema Complementario tiene la menor prioridad porque su uso es ocasional y su impacto más acotado.

Para recuperación ante desastres (DR) se cuenta con una región de respaldo geográficamente distinta dentro de AWS, con backups regulares de las bases de datos, y con un RTO (tiempo de recuperación) y RPO (pérdida de datos máxima tolerable)

definidos, para que el sistema pueda volver a operar rápido si llegara a caerse toda una región.

Para el monitoreo se usa tracing distribuido con AWS X-Ray, que permite seguir el rastro de una petición del usuario a través de los distintos microservicios por los que pasa, esto es importante porque en una arquitectura de microservicios una sola petición puede pasar por 4 o 5 servicios distintos y hay que poder identificar en cuál falló. Esto se complementa con métricas de CPU, latencia y errores, y logs centralizados mediante CloudWatch.

Por último, para el auto-healing se usa un orquestador de contenedores (Kubernetes, vía Amazon EKS) que hace chequeos de salud (health checks) periódicos a cada instancia, si una deja de responder la reemplaza automáticamente sin intervención humana, y además maneja auto-scaling según la carga del sistema.

Manejo de Costos

La estrategia de costos combina varios enfoques según el tipo de carga de cada componente. Para la carga base constante, como los nodos de EKS y las instancias de ElastiCache que corren 24/7, se usan instancias reservadas (Reserved Instances/Savings Plans), que salen más baratas a largo plazo que pagar por demanda todo el tiempo. Para los picos de demanda variables, como las horas de mayor tráfico de transacciones, se usa auto-scaling, así solo se paga de más cuando realmente hay más carga, en vez de sobreaprovisionar todo el sistema al nivel de la hora pico durante todo el día.

La base de datos de auditoría (DynamoDB) se configura en modo de facturación bajo demanda (on-demand) en vez de capacidad aprovisionada, ya que el volumen de eventos puede variar bastante y es difícil predecirlo con exactitud, este modo evita pagar por capacidad reservada que no se use.

Los videos de verificación facial subidos a S3 durante el onboarding se gestionan con políticas de ciclo de vida (lifecycle policies), moviéndolos automáticamente a un almacenamiento más económico después de un periodo de retención definido, o eliminándolos según lo que indique la normativa de protección de datos aplicable.

Para la visibilidad y control de gastos se usa AWS Cost Explorer, con etiquetado de recursos por servicio para poder identificar qué microservicio o componente está generando más costo y ajustar en consecuencia.

Conclusiones

La arquitectura propuesta para el sistema de banca por internet de BP se diseñó bajo el modelo C4, con los 3 niveles completos (Contexto, Contenedores y Componentes) y cada decisión arquitectónica justificada con al menos 2 razones teóricas, cubriendo todos los requerimientos del enunciado, desde la integración con el Core y el Sistema Complementario, hasta el onboarding con reconocimiento facial, la autenticación OAuth2.0, el sistema de notificaciones, la auditoría y el despliegue en AWS.

A lo largo del diseño se priorizó la seguridad y la tolerancia a fallos sobre la simplicidad, reflejado en decisiones como el patrón BFF para proteger los tokens de la SPA, el Almacén de Sesiones dedicado separado de la caché general, la Firma de Transacción con Dynamic Linking, y el Circuit Breaker priorizado por radio de impacto en cada dependencia externa. También se buscó mantener una arquitectura desacoplada y reutilizable, reflejado en el uso consistente de patrones como Adapter, Aggregator, Cache-Aside y Pub/Sub a través de distintos microservicios, en vez de soluciones aisladas para cada problema.

Con esto, la solución queda lista para operar de forma segura, escalable y mantenible a futuro, cumpliendo con los requerimientos técnicos y normativos propios de una entidad financiera.